BẢNG PHÂN CÔNG CHI TIẾT TUẦN 2

Tích Hợp ALNS (Adaptive Large Neighborhood Search)

Lưu ý về đánh số tuần: Theo tài liệu kế hoạch gốc, Tuần 1 (Local Search — `operators.py, evaluator.py, hill_climbing.py`) vừa hoàn thành. Tuần 2 (05/10–12/10) là Tích hợp ALNS — nội dung chính của tài liệu này. Tuần 3 (13/10–20/10) là Tinh chỉnh tham số & Tối ưu tốc độ, chỉ bắt đầu có ý nghĩa sau khi ALNS đã chạy được — xem tóm tắt ở mục 5.

Làm việc song song, giống mô hình Tuần 1.

1. Cơ chế hoạt động của ALNS (cả 2 người cần hiểu trước khi code)

ALNS là vòng lặp Destroy → Repair, chạy trên nền nghiệm đã có (đầu ra của Hill Climbing):

Vòng lặp ALNS

```
current_solution = kết quả Hill Climbing
best_solution = current_solution
while chưa hết ngân sách (số vòng / thời gian):
    1. Chọn 1 destroy operator (theo trọng số)
       -> rút q đơn ra khỏi current_solution
    2. Chọn 1 repair operator (theo trọng số)
       -> chèn lại q đơn đó
    3. new_solution = kết quả sau destroy + repair
    4. Nếu new_solution tốt hơn best_solution -> cập nhật best_solution
    5. Accept/Reject new_solution làm current_solution cho vòng sau
       (accept cả khi tệ hơn, theo 1 tiêu chí xác suất - vd: SA)
    6. Cập nhật điểm thưởng (score) cho destroy/repair operator
       vừa dùng, dựa trên kết quả
    7. Sau mỗi K vòng: cập nhật lại trọng số chọn operator
       dựa trên điểm thưởng tích lũy
return best_solution
```

Khác Hill Climbing ở đâu: Hill Climbing chỉ dịch chuyển tại chỗ 1 khối trong route hiện có (không bao giờ rút hẳn nhiều đơn ra cùng lúc). ALNS rút hẳn 1 cụm đơn (q có thể là 10-30% tổng số đơn) ra khỏi mọi route, rồi xây lại từ đầu cụm đó — nên phá được cấu trúc cũ triệt để hơn, tránh bị kẹt ở 1 local optimum mà Hill Climbing không thoát ra được.

2. Nhánh 1: Destroy Operators (Hà Thu)

Mục tiêu: Viết các hàm "rút đơn ra"khỏi nghiệm hiện tại — vẫn là xử lý mảng thuần túy, đúng vai trò bạn đang làm ở `operators.py`.

File làm việc: `destroy_operators.py` (tạo mới, KHÔNG sửa `operators.py` cũ — file đó vẫn được `hill_climbing.py` dùng, đừng động vào).

Các hàm cần hoàn thiện:

- `random_destroy(routes, q, instance)`
  → `Tuple[routes_moi, removed_requests]`
  Chọn ngẫu nhiên q đơn (khách hoặc hàng) đang được phục vụ trong routes, rút hẳn khối của chúng ra (dùng lại đúng logic "xác định block"đã viết trong `_get_request_block` của `operators.py`, có thể copy/export ra dùng chung). Trả về route đã rút xong + danh sách các `PassengerRequest`/`ParcelRequest` đã bị rút (để Dũng chèn lại ở bước repair).

- `worst_removal_destroy(routes, q, instance, removal_cost_fn)`
  → `Tuple[routes_moi, removed_requests]`
  Rút q đơn có "chi phí đóng góp cao nhất"(tức route sẽ rẻ đi nhiều nhất nếu bỏ đơn đó) — ĐÂY LÀ CHỖ CẦN GHÉP VỚI DŨNG: nhận vào 1 hàm `removal_cost_fn` (Dũng viết, đặt trong `evaluator.py`, tái dùng chính công thức `calculate_delta_cost` / `calculate_pair_delta_cost` vừa viết cho Hill Climbing — vì "tiết kiệm khi rút 1 khối"chính là công thức đó). Bạn CHỈ CẦN GỌI hàm này, không cần biết công thức bên trong. Sắp toàn bộ đơn đang phục vụ theo `removal_cost_fn` giảm dần, rút q đơn đầu (có thêm chút nhiễu ngẫu nhiên khi chọn, để tránh luôn rút đúng 1 nhóm y hệt mỗi lần).

- `shaw_destroy(routes, q, instance, relatedness_fn)`
  → `Tuple[routes_moi, removed_requests]`
  Rút q đơn "có liên quan"đến nhau (gần nhau về không gian/thời gian) — ý tưởng: xoá đi 1 cụm việc "cùng khu vực"thì repair dễ tìm ra cách sắp xếp lại tốt hơn là rút ngẫu nhiên. Chọn 1 đơn hạt giống (seed) ngẫu nhiên, rồi lặp: tính độ liên quan giữa đơn hạt giống và mọi đơn còn lại bằng hàm `relatedness_fn` (Dũng viết, dựa trên `cost_matrix`/`time_matrix`), rút những đơn liên quan nhất.

Interface chốt (giống style Tuần 1): cả 3 hàm destroy đều trả về đúng dạng (`routes_moi: List[List[Node]], removed: List[Union[PassengerRequest, ParcelRequest]]`) để Dũng cắm thẳng vào bước repair, không cần đoán.

3. Nhánh 2: Repair Operators + Lõi ALNS (Dũng)

Mục tiêu: Chèn lại các đơn đã bị rút, và viết vòng lặp ALNS hoàn chỉnh (thích ứng trọng số + tiêu chí accept/reject) — đúng vai trò "toán học tối ưu"bạn đang làm.

File làm việc 1: `evaluator.py` (mở rộng, KHÔNG xoá gì đang có)

- `calculate_removal_saving(routes, request, instance)` → `float`
  Đây chính là hàm `removal_cost_fn` mà Hà Thu cần gọi ở `worst_removal_destroy`. Tái

dùng trực tiếp `calculate_delta_cost` (đơn bưu kiện, 1 node) hoặc `calculate_pair_delta_cost` (đơn khách, cặp 2 node) đã viết cho `hill_climbing.py` — tìm request đang ở route/vị trí nào, lấy 2 nút biên A, B, gọi đúng công thức đã có. KHÔNG viết công thức mới.

- `calculate_relatedness(req_a, req_b, instance)` → `float`
  Công thức chuẩn trong ALNS: relatedness thấp = liên quan nhiều = ưu tiên rút cùng nhau.
  Ví dụ đơn giản: $relatedness = cost(pickup_a, pickup_b) + cost(dropoff_a, dropoff_b)$ (khoảng cách không gian giữa 2 đơn) — có thể cộng thêm phần chênh giờ đón nếu muốn tinh hơn.

File làm việc 2: `repair_operators.py` (tạo mới)

- `greedy_repair(routes, removed_requests, instance)` → `routes_moi`
  Với từng đơn bị rút, tìm vị trí chèn lời nhất trên MỌI xe (tái dùng logic `find_best_passenger_insertion` / `find_best_parcel_insertion` của `greedy_solver.py` — gọi lại được vì đó đã là method của `GreedySolver`, có thể tách thành hàm độc lập dùng chung nếu cần, KHÔNG viết lại từ đầu). Nếu 1 đơn không tìm được vị trí nào lời (gain > 0) thì bỏ qua (không bắt buộc phục vụ hết).

- `regret_insertion(routes, removed_requests, instance)` → `routes_moi`
  Với mỗi đơn bị rút, tính chi phí chèn vào xe TỐT NHẤT và xe TỐT NHÌ. "Regret-chênh lệch giữa 2 chi phí đó. Chèn đơn có regret CAO NHẤT trước (đơn mà nếu trễ mới chèn sẽ mất nhiều cơ hội nhất, vì xe tốt nhất của nó có thể bị đơn khác chiếm trước) — tinh hơn Greedy Repair (Greedy Repair chỉ nhìn chi phí tuyệt đối, không nhìn "cơ hội bị mất").

File làm việc 3: `alns.py` (tạo mới, chứa class `ALNSSolver`)

- Nhận đầu vào: nghiệm cơ sở từ `HillClimbingSolver` (giống cách `HillClimbingSolver` nhận từ `GreedySolver`).
- Vòng lặp chính (đúng 7 bước ở mục 1), có:
  - Danh sách trọng số cho destroy operators và repair operators riêng (roulette wheel).
  - Tiêu chí accept: đơn giản nhất là "chỉ nhận khi tốt hơn hoặc bằng current"(an toàn, giống Hill Climbing); nâng cấp thêm là chấp nhận cả nghiệm tệ hơn với xác suất giảm dần theo "nhiệt độ"(Simulated Annealing) — CÓ THỂ để phiên bản đơn giản trước, thêm SA sau nếu còn thời gian (đây chính là việc Tuần 3 sẽ tinh chỉnh).
  - Ngưỡng dừng: số vòng tối đa hoặc time-limit (nên có time-limit cứng ngay từ đầu, tránh bị timeout khi nộp bài).

4. Giao diện (Interface) chốt trước — để 2 người không phải chờ nhau

Hà Thu khi viết `worst_removal_destroy` / `shaw_destroy` cứ gọi thẳng `removal_cost_fn` / `relatedness_fn` như tham số truyền vào (chưa cần Dũng viết xong nội dung thật —

Hàm xử lý	Phân công	Input	Output
`random_destroy`	Hà Thu	`routes, q, instance`	`(routes_moi, removed)`
`worst_removal_destroy`	Hà Thu	`routes, q, instance, removal_cost_fn`	`(routes_moi, removed)`
`shaw_destroy`	Hà Thu	`routes, q, instance, relatedness_fn`	`(routes_moi, removed)`
`calculate_removal_saving`	Dũng	`routes, req, instance`	`float`
`calculate_relatedness`	Dũng	`req_a, req_b, instance`	`float`
`greedy_repair`	Dũng	`routes, removed, instance`	`routes_moi`
`regret_insertion`	Dũng	`routes, removed, instance`	`routes_moi`
`ALNSSolver.solve()`	Dũng	(dùng các hàm trên)	`routes_moi, best_benefit`

có thể tự mock 1 hàm trả số ngẫu nhiên để test luồng rút đơn của mình trước, đúng tinh thần "test độc lập"đã làm ở Tuần 1).

5. Quy trình ghép code (giống Tuần 1)

1. Dũng viết `calculate_removal_saving` / `calculate_relatedness` trong `evaluator.py` trước (không phụ thuộc destroy operators), để Hà Thu có hàm thật để gọi càng sớm càng tốt.
2. Hà Thu viết 3 hàm destroy trong `destroy_operators.py`, tự test bằng `is_route_valid` sau khi rút (route còn lại phải vẫn hợp lệ — vì rút đúng "cả khối"không được để dở dang giữa cặp khách).
3. Dũng viết `greedy_repair`, `regret_insertion`, và `ALNSSolver` trong `repair_operators.py` / `alns.py`.
4. **Ráp:** `main.py` thêm `demo_alns()` — chạy `ALNSSolver` trên nền `HillClimbingSolver`, in Benefit trước/sau, xác nhận Benefit không giảm và mọi route cuối qua `is_route_valid` (đúng cách đã làm với `demo_hill_climbing`).
5. Test trên bộ dữ liệu nhỏ trước khi chạy benchmark lớn — so Benefit ALNS vs Hill Climbing thuần (ALNS phải bằng hoặc tốt hơn, vì nó chạy TIẾP sau Hill Climbing).